ASP.NET Web Development

Unit 8: Creating and Consuming Web Services

Topics: XML Web Services • SOAP • WSDL • UDDI • Web Service Consumers

---

1. The Motivation for XML Web Services

1.1 What is a Web Service?

A Web Service is a software component that can be accessed over a network (typically the internet) using standard protocols. It allows applications written in different languages, running on different platforms, to communicate and share data seamlessly.

XML Web Service: A component accessible via HTTP that uses XML to exchange data, making it platform and language independent

SOAP: Simple Object Access Protocol — the XML-based messaging protocol used by Web Services

WSDL: Web Services Description Language — XML file describing the web service interface

UDDI: Universal Description, Discovery and Integration — a directory for discovering web services

1.2 Why Do We Need Web Services?

Problems Before Web Services:

- Applications on different platforms could not easily communicate
- No standard way to expose business logic over the internet
- RPC (Remote Procedure Call) and DCOM were platform-dependent
- Firewalls blocked most protocols except HTTP (port 80)
- Binary data formats were not interoperable

How Web Services Solve These Problems:

- Use HTTP — universally allowed through firewalls
- Use XML — human-readable, platform-neutral data format
- WSDL provides a self-describing interface — clients auto-generate proxies
- Language independence: Java client can call a C# web service
- Enables Service-Oriented Architecture (SOA)

1.3 Real-World Use Cases

Use Case	Web Service Provider	Consumer
Payment Processing	PayPal / Stripe API	E-commerce website
Weather Data	Weather.com service	Mobile app
Currency Conversion	Bank's web service	Travel booking portal
SMS Notifications	Twilio service	Any application
Map & Location	Google Maps API	Delivery tracking app

> 📌 **Exam Tip:** Web Services are the foundation of modern APIs and microservices architecture. Understanding them is essential for enterprise application development.

1.4 Web Service Architecture Overview

The Web Service architecture involves three key roles:

- Service Provider: Creates and publishes the web service
- Service Registry (UDDI): Stores descriptions of available services
- Service Consumer: Discovers and uses the web service

The three key operations are:

- Publish: Provider registers the service in UDDI registry
- Find: Consumer searches the UDDI registry for required service
- Bind: Consumer invokes the service using WSDL information

1.5 Web Services vs Traditional Web Applications

Feature	Traditional Web App	XML Web Service

Output	HTML for browsers	XML data for applications
Consumer	Human users	Other applications/programs
Protocol	HTTP + HTML	HTTP + SOAP/XML
Interface	Visual UI	WSDL contract
Language dependency	Yes (mostly)	No — language independent
Platform dependency	Often platform-tied	Fully platform independent

2. Creating an XML Web Service with Visual Studio

2.1 Setting Up a Web Service Project

In Visual Studio, a Web Service project is created using the ASP.NET Web Service Application template. Each web service is a .asmx file.

Steps to Create a Web Service:

- Open Visual Studio → File → New → Project
- Select: ASP.NET Web Application or Web Service
- Add a new item: Web Service (.asmx)
- Write web methods inside the service class
- Test using the browser (Visual Studio provides a test page)

2.2 The .asmx File

Web services in ASP.NET use the .asmx extension. The service directive at the top identifies it as a web service:

```
<%@ WebService Language="C#" CodeBehind="MathService.asmx.cs"
    Class="WebServiceDemo.MathService" %>
```

2.3 Creating a Simple Web Service

Code-Behind (MathService.asmx.cs):

```
using System;
using System.Web.Services;

namespace WebServiceDemo
{
    [WebService(Namespace = "http://tempuri.org/",
                Description = "A simple Math Web Service")]
    [WebServiceBinding(ConformsTo = WsiProfiles.BasicProfile1_1)]
    public class MathService : System.Web.Services.WebService
    {
        [WebMethod(Description = "Adds two integers")]
        public int Add(int a, int b)
        {
            return a + b;
```

```
        }

        [WebMethod(Description = "Multiplies two numbers")]
        public double Multiply(double x, double y)
        {
            return x * y;
        }

        [WebMethod]
        public string Greet(string name)
        {
            return "Hello, " + name + "!";
        }
    }
}
```

2.4 Important Attributes

Attribute	Target	Purpose
[WebService]	Class	Marks class as a web service; sets namespace, description
[WebMethod]	Method	Exposes method as a callable web service operation
[WebServiceBinding]	Class	Specifies SOAP binding conformance profile
Description	WebMethod param	Adds documentation to the method in WSDL
MessageName	WebMethod param	Overloads methods with same name
CacheDuration	WebMethod param	Caches results for given seconds
EnableSession	WebMethod param	Enables session state for the method

> 📌 **Exam Tip:** [WebMethod] is mandatory — only methods with this attribute are exposed as web service operations.

2.5 Testing a Web Service

When you navigate to a .asmx file in a browser, ASP.NET provides an automatic test page that allows you to:

- View all available web methods
- View the WSDL description of the service
- Invoke individual methods and see the XML response
- View SOAP request and response message formats

URL to access test page: http://localhost:port/MathService.asmx

URL to access WSDL: http://localhost:port/MathService.asmx?WSDL

3. Designing XML Web Services

3.1 Design Principles

- Keep methods coarse-grained — one call should accomplish a complete task
- Use meaningful namespaces (not the default http://tempuri.org/)
- Document all methods using Description property of [WebMethod]
- Design methods to be stateless wherever possible
- Return simple types or XML-serializable complex types
- Handle exceptions gracefully — return meaningful error messages

3.2 SOAP — Simple Object Access Protocol

SOAP is the XML-based protocol used to send and receive web service messages. Every web service call is wrapped in a SOAP envelope.

SOAP Message Structure:

```
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header>
    <!-- Optional: authentication, metadata -->
  </soap:Header>
  <soap:Body>
    <!-- Required: actual request/response data -->
    <Add xmlns="http://tempuri.org/">
      <a>5</a>
      <b>3</b>
    </Add>
  </soap:Body>
</soap:Envelope>
```

SOAP Response:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <AddResponse xmlns="http://tempuri.org/">
      <AddResult>8</AddResult>
    </AddResponse>
  </soap:Body>
```

```
</soap:Envelope>
```

3.3 WSDL — Web Services Description Language

WSDL is an XML document that describes the web service — what it does, how to call it, what parameters it expects, and what it returns. It is the contract between the service provider and consumer.

WSDL Main Elements:

WSDL Element	Purpose
<types>	Defines data types used by the service (XML Schema)
<message>	Defines the data elements for each operation (input/output)
<portType>	Defines the operations (methods) the service supports
<binding>	Specifies SOAP protocol details for each operation
<service>	Specifies the URL (endpoint) where the service is available

> 📌 **Exam Tip:** You don't write WSDL manually — ASP.NET generates it automatically. Access it by appending ?WSDL to the service URL.

3.4 Using Complex Types in Web Services

Web services can return complex objects (classes) as long as they are XML-serializable:

```
// Define a serializable class
[Serializable]
public class Student
{
    public int ID { get; set; }
    public string Name { get; set; }
    public double CGPA { get; set; }
}

// Web method returning a complex type
[WebMethod]
public Student GetStudent(int id)
```

```
{
    return new Student { ID = id, Name = "Alice", CGPA = 9.2 };
}

// Web method returning a list
[WebMethod]
public List<Student> GetAllStudents()
{
    // Fetch from database and return
    return studentList;
}
```

3.5 WebMethod Properties Reference

Property	Type	Default	Description
Description	string	""	Documents the method in WSDL and test page
EnableSession	bool	false	Enables HttpSession for the web method
CacheDuration	int	0	Seconds to cache response (0 = disabled)
MessageName	string	method name	Enables overloading of web method names
TransactionOption	enum	Disabled	Transaction support (Supported, Required, etc.)
BufferResponse	bool	true	Buffer entire response before sending

4. Creating Web Service Consumers

4.1 What is a Web Service Consumer?

A Web Service Consumer (also called a client) is any application that calls and uses a web service. The consumer uses a proxy class to communicate with the web service — it calls the proxy as if it were a local class, and the proxy handles all the SOAP communication.

4.2 Adding a Web Reference in Visual Studio

Steps to Add a Web Reference (.NET Framework):

- Right-click the project in Solution Explorer → Add Service Reference
- Click 'Advanced' → 'Add Web Reference' (for older style)
- Enter the WSDL URL: http://server/MathService.asmx?WSDL
- Click Go → Visual Studio discovers available methods
- Enter a namespace (e.g., MathServiceRef)
- Click Add Reference → proxy class is generated automatically

> 📌 **Exam Tip:** Visual Studio generates a proxy class from the WSDL. This proxy contains local method signatures that match the web service operations.

4.3 The Proxy Class

When you add a web reference, Visual Studio creates a proxy class (Reference.cs) that:

- Has the same method signatures as the web service
- Handles SOAP envelope creation and parsing
- Manages HTTP communication to the service endpoint
- Converts return values back to .NET types

4.4 Calling a Web Service from a Consumer

Synchronous Call (simple approach):

```
// In your ASP.NET page or Windows application
using MathServiceRef; // namespace from Add Reference
```

```
protected void btnCalculate_Click(object sender, EventArgs e)
{
    // Create proxy object
    MathService proxy = new MathService();

    // Call web method as if it were a local method
    int result = proxy.Add(10, 25);
    lblResult.Text = "Sum: " + result.ToString();

    string greeting = proxy.Greet("Engineering Student");
    lblGreet.Text = greeting;
}
```

Asynchronous Call (recommended for UI responsiveness):

```
protected void btnAsync_Click(object sender, EventArgs e)
{
    MathService proxy = new MathService();
    // BeginXxx / EndXxx pattern
    IAsyncResult ar = proxy.BeginAdd(10, 25, null, null);
    int result = proxy.EndAdd(ar);
    lblResult.Text = result.ToString();
}
```

4.5 Handling Errors in Web Service Consumers

```
try
{
    MathService proxy = new MathService();
    int result = proxy.Add(10, 20);
    lblResult.Text = result.ToString();
}
catch (System.Web.Services.Protocols.SoapException soapEx)
{
    // SOAP-level errors (thrown by the service itself)
    lblError.Text = "Service Error: " + soapEx.Message;
}
catch (System.Net.WebException netEx)
{
    // Network errors (service unavailable, timeout)
    lblError.Text = "Network Error: " + netEx.Message;
}
```

> 📌 **Exam Tip:** Always handle both SoapException (service logic errors) and WebException (network/connectivity errors) when consuming web services.

4.6 Complete Consumer Example — ASP.NET Page

ASPX Markup:

```
<asp:TextBox ID="txtA" runat="server" />
<asp:TextBox ID="txtB" runat="server" />
<asp:Button ID="btnAdd" runat="server" Text="Add"
    OnClick="btnAdd_Click" />
<asp:Label ID="lblResult" runat="server" />
```

Code-Behind:

```
protected void btnAdd_Click(object sender, EventArgs e)
{
    try {
        MathServiceRef.MathService svc =
            new MathServiceRef.MathService();
        int a = int.Parse(txtA.Text);
        int b = int.Parse(txtB.Text);
        lblResult.Text = "Result: " + svc.Add(a, b);
    }
    catch (Exception ex) {
        lblResult.Text = "Error: " + ex.Message;
    }
}
```

5. Discovering Web Services Using UDDI

5.1 What is UDDI?

UDDI (Universal Description, Discovery and Integration) is a platform-independent, XML-based registry for businesses to list their web services. It acts as a 'yellow pages' or 'telephone directory' for web services on the internet.

UDDI: A global directory where businesses publish and discover web services using standardized XML structures

UDDI Registry: A web-based database that stores descriptions of web services and the businesses that provide them

Business Entity: A company or organization registered in UDDI

Business Service: A web service offered by a business entity in UDDI

Binding Template: Technical details (URL, WSDL) for accessing a web service in UDDI

5.2 UDDI Data Structures

Structure	Equivalent to	Contains
businessEntity	White pages	Company name, description, contacts
businessService	Yellow pages	Service category and description
bindingTemplate	Green pages	Technical access point (URL, WSDL)
tModel	Technical specification	Reusable technical metadata / interface definition

5.3 The Three 'Pages' of UDDI

- White Pages: Basic business information — name, address, phone, contact details
- Yellow Pages: Business category classification — industry type, location, services
- Green Pages: Technical details — how to invoke the service, WSDL location, binding info

5.4 UDDI Discovery Process

Step-by-Step Flow:

- 1. Provider: Develop and deploy the web service
- 2. Provider: Publish service details in UDDI registry (businessEntity + businessService + bindingTemplate)
- 3. Consumer: Search UDDI registry by business name, category, or tModel
- 4. Consumer: Retrieve bindingTemplate to get the WSDL URL
- 5. Consumer: Download WSDL and generate a proxy class
- 6. Consumer: Call the web service using the proxy

> 📌 **Exam Tip:** The UDDI → WSDL → Proxy chain is the standard discovery-to-consumption pipeline. Know this sequence for exams.

5.5 Accessing UDDI in Visual Studio

Visual Studio provides built-in UDDI integration through the 'Add Web Reference' dialog:

- 'UDDI Directory' option searches the Microsoft UDDI Business Registry
- 'Test Microsoft UDDI Directory' for private intranet registries
- Results show businesses and their available web services
- You can directly add a web reference from the UDDI search results

5.6 Private UDDI Registries

Large organizations often set up private UDDI registries for internal use:

- Allows enterprises to maintain an internal directory of web services
- Windows Server includes a built-in UDDI Services component
- Accessible via http://server/uddi/ on the intranet
- Useful for SOA governance within an organization

5.7 UDDI vs Other Discovery Mechanisms

Mechanism	Standard	Use Case
UDDI	OASIS Standard	Enterprise/B2B web service discovery
DISCO (.disco file)	Microsoft proprietary	Simple ASP.NET service discovery on same server

WS-Inspection (WSIL)	IBM/Microsoft	Lightweight service inspection/discovery
Direct WSDL URL	W3C Standard	When you already know the service endpoint

5.8 DISCO — Simple Discovery Alternative

DISCO (Web Services Discovery) is a simpler Microsoft-specific discovery mechanism. A .disco file is an XML document that lists available web services on a server.

```
<!-- Example: default.disco -->
<?xml version="1.0" ?>
<disco:discovery xmlns:disco="http://schemas.xmlsoap.org/disco/"
                 xmlns:scl="http://schemas.xmlsoap.org/disco/scl/">
  <scl:contractRef ref="http://server/MathService.asmx?WSDL" />
  <scl:contractRef ref="http://server/StudentService.asmx?WSDL" />
</disco:discovery>
```

Summary: Key Concepts Quick Reference

Topic	Key Terms	Important Points
Web Services	SOAP, WSDL, UDDI	Platform/language independent, uses HTTP+XML
Creating Service	[WebService], [WebMethod]	.asmx file, namespace required, auto WSDL
SOAP	Envelope, Header, Body	XML messaging protocol for web service calls
WSDL	types, message, portType, binding	Auto-generated contract; access via ?WSDL
Consumer	Proxy class, Add Reference	Proxy handles SOAP; call methods as local
Error Handling	SoapException, WebException	Handle both types in consumer code
UDDI	White/Yellow/Green pages	Directory for discovering web services
DISCO	.disco file	Simple MS-specific service discovery